

Evaluating the Effect of Text Preprocessing on Traditional, Deep Learning, and Transformer Models for Toxic Comment Detection

Tanisha Taranoon Hridy
tanishataranoonhridy@gmail.com
Dept. of CSE,
University of Asia Pacific

Abstract—With increasing usage of online communication platforms, toxic comments become a widespread problem that negatively impacts both user experience and digital safety [1]. Detection of toxic comments poses a complicated task in NLP because of informality of language and ambiguous context. This research explores how the application of several preprocessing methods influences the performance of NLP model architectures for toxic comment classification [2]. The paper provides a comparative analysis of Machine Learning, Deep Learning and Transformer-based approaches for toxic comment classification based on Jigsaw Toxic Comment Classification dataset [3]. While most existing studies focus on the accuracy of the proposed models, this research highlights the importance of the relationship between preprocessing and models [4]. The results were evaluated using accuracy, precision, recall and macro F1-score metrics. Among all examined approaches, the best overall results were produced by the combination of TF-IDF + SVM approach trained on raw text, achieving the accuracy of 92.09% and macro F1-score of 0.5944. Although both Deep Learning and Transformer-based models showed the ability to grasp context, they required much more computational power and were not superior to SVM baseline in the given experiments. Thus, carefully crafted traditional Machine Learning methods prove to be very efficient at toxic comment classification task, along with being computationally effective.

Index Terms—Toxic comment detection, NLP, TF-IDF, BiLSTM, BERT, preprocessing impact, model comparison, text classification.

I. INTRODUCTION

Due to increasing popularity of online communication platforms, toxic content becomes more common, including insult language, abuse and hate speech [5]. Automatic detection of toxic content is crucial but complicated because of unstructured informal text [6]. This project explores different approaches for toxic text classification and the effect of preprocessing on their performance. The objective of this research is to compare not only classification accuracy but also the preprocessing sensitivity and computational efficiency of different architectures.

II. OBJECTIVE

The objectives of this project are as follows:

- To compare traditional machine learning, deep learning, and transformer-based models for toxic text classification.
- To analyze the impact of different preprocessing techniques on model performance.
- To evaluate whether raw text or cleaned text performs better across different architectures.
- To compare models using multiple evaluation metrics and computational efficiency.

III. DATASET AND PREPROCESSING:

A. Dataset Description

In this work, the Jigsaw Toxic Comment Classification dataset [3] is used, which is one of the most popular datasets for detecting toxic language and represents the comments from Wikipedia annotated for several toxicity classes. It is composed of comments in English language that are gathered in online discussions and labeled into several categories of toxicity.

1) Key characteristics of the dataset include::

- Large-scale English text data
- Multi-label annotations (toxic, insult, threat, etc.)
- Informal noisy user-generated content
- Possible class imbalance
- The dataset has 8 columns (features) in the main training file
 - 1 input feature → comment text
 - 6 output labels → multi-label classification
- Number of rows
 - training set: 160,000 rows
 - total number of rows: 465,899 rows

2) *Toxicity Labels*: The dataset contains toxicity classes include the following:

- toxic
- severe-toxic
- obscene
- threat
- insult
- identity-hate

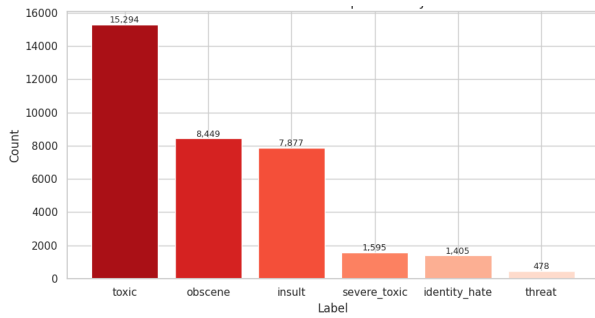


Fig. 1. Number of comments per Toxicity Label

B. Exploratory Data Analysis

Exploratory data analysis (EDA) is performed in order to get a more detailed knowledge about the features of the dataset, such as:

- Label distribution analysis
- Visualization of the toxic/non-toxic ratio

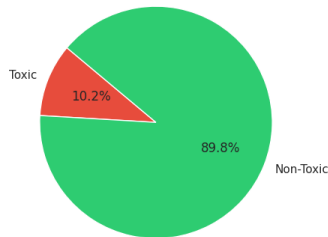


Fig. 2. Toxic VS Non-Toxic

- Word frequency analysis
- N-gram analysis

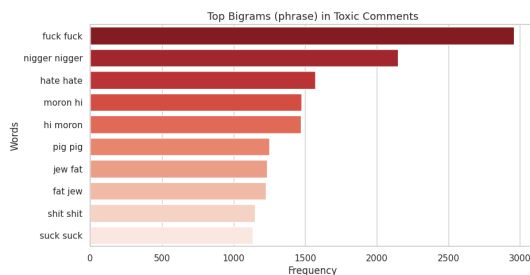


Fig. 3. Bigrams

- Word cloud



Fig. 4. Toxic Comment Word Cloud

- Co-occurrence matrix visualization
- Text length distribution

C. Text Preprocessing

Preprocessing of text is performed for improving its quality and performance of the future machine learning model. Three versions of the dataset are prepared, namely: raw texts, basic clean text and advanced cleaned texts.

1) *Basic Preprocessing:*

- Lowercasing text
- Removing URLs and user mentions
- Removing punctuation
- Punctuation removal
- Extra whitespace removal

2) Advanced Preprocessing:

- Emoji to text conversion
- Contraction expansion
- Obscene word normalization
- Repeated character normalization
- Tokenization
- Stopword removal
- Lemmatization

The mentioned steps are taken for reducing the noise of texts and improving the quality of inputs.

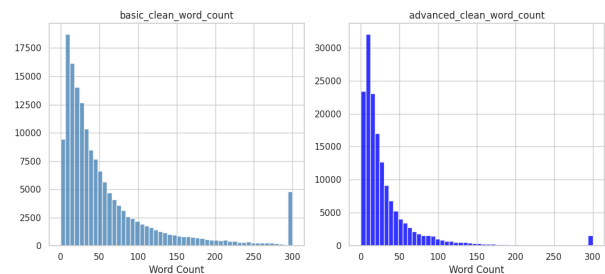


Fig. 5. Impact of both Preprocessing

IV. SYSTEM DESIGN AND METHODOLOGY

A. Overall Workflow

Proposed workflow includes the following steps:

It allows for a comparative analysis of different models in the same experimental settings. Block diagram of the proposed methodology is provided in 6.

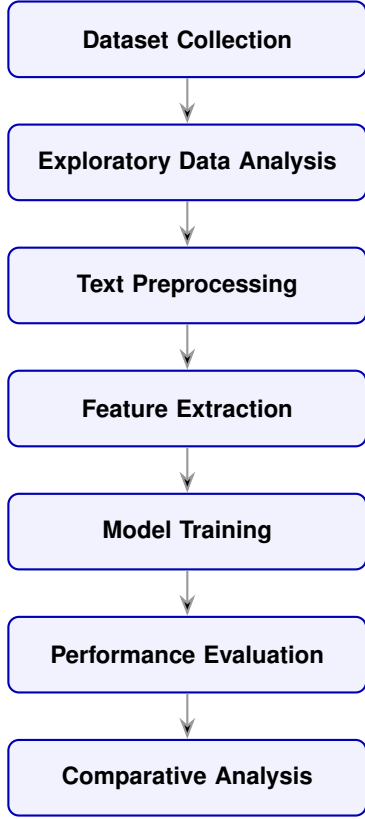


Fig. 6. Block diagram of the proposed sequential methodology.

B. Feature Extraction

As different model architectures mean that there is a need for different kinds of inputs, different techniques for feature extraction has been applied.

1) **TF-IDF Vectorization:** Preprocessed text data is mapped to feature vectors using the technique of Term Frequency – Inverse Document Frequency (TF-IDF). TF-IDF gives high weight values to informative terms and low to commonly found words [7].

- **Sublinear Term Frequency (TF):** As sublinear $tf = \text{True}$, the term frequencies have been mapped on the logarithmic scale:

$$TF(t, d) = 1 + \log(f_{t,d}) \quad (1)$$

where:

- $f_{t,d}$ = frequency of term t in document d

It helps to diminish the contribution of very frequently occurring words and not allow them to dominate the feature space.

- **Inverse Document Frequency (IDF):** The importance of each term in the entire text corpus is captured by the Inverse Document Frequency:

$$IDF(t) = \log\left(\frac{N}{DF(t)}\right) \quad (2)$$

where:

- N = total number of documents
- $DF(t)$ = number of documents containing term t

Common terms get low IDF scores, whereas rare terms get higher IDF scores.

- **TF-IDF Weight:** The final TF-IDF score is computed as:

$$TF-IDF(t, d) = TF(t, d) \times IDF(t) \quad (3)$$

The TF-IDF vectorizer was configured with a maximum vocabulary of size 50,000 features, both unigrams and bigrams, as well as a minimum document frequencies have been considered. Thus obtained sparse feature vectors are passed as the input to the Logistic Regression and Support Vector Machine classifiers.

2) **Sequence Encoding:** Deep learning algorithms cannot take raw texts as input. Hence, textual comments have been encoded to numerical sequences using tokenization and integer encoding.[8]

Assume a comment containing a sequence of words:

$$X = \{w_1, w_2, w_3, \dots, w_n\} \quad (4)$$

each word is mapped to a unique integer index:

$$X_{\text{enc}} = \{i_1, i_2, i_3, \dots, i_n\} \quad (5)$$

where i_k represents the vocabulary index of word w_k .

Since comments vary in length, post-padding is applied to obtain a uniform fixed-length sequence of dimension L :

$$X_p = [i_1, i_2, \dots, i_n, 0, \dots, 0] \in \mathbb{R}^L \quad (6)$$

where zeros represent padding tokens appended to shorter sequences to match the maximum sequence length.

The encoded sequences are subsequently transformed into continuous, dense vector representations through a trainable embedding layer:

$$\mathbf{e}_t = \mathbf{E}(i_t) \quad (7)$$

where:

- $\mathbf{E} \in \mathbb{R}^{V \times D}$ represents the embedding matrix weight (where V is vocabulary size and D is embedding dimension).
- i_t denotes the token index at time step t .
- $\mathbf{e}_t \in \mathbb{R}^D$ denotes the resulting dense embedded vector representation.

The sequence of embedding vectors $\mathbf{X}_e = (\mathbf{e}_1, \mathbf{e}_2, \dots, \mathbf{e}_L)$ serves directly as the input sequence to the BiLSTM network.

3) **Transformer-Based Encoding (DistilBERT):** The transformer architecture employs a scaled dot-product self-attention scheme to understand bidirectional contextual connections between words in a sequence.[9]

The attention scheme can be written mathematically as: The

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right) \mathbf{V} \quad (8)$$

where:

- $\mathbf{Q}$ = Query matrix

- $\mathbf{K}$ = Key matrix
- $\mathbf{V}$ = Value matrix
- d_k = Dimensionality of the key vectors

This allows the transformer to compute dynamic weights and figure out what words are important to one another when creating deep contextual representations. In order to generate output probabilities for multi-label toxic In comments classification, the sigmoid activation function is used for all output classes.

$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad (9)$$

where z represents the logit output computed by the final fully connected classification layer.

Consequently, each toxicity label is assigned an independent probability score bounded between 0 and 1, allowing for the simultaneous detection of multiple toxic attributes.

C. Model Selection

Three groups of models have been implemented to evaluate models' performance depending on architectures.

1) Traditional Machine Learning Models:

- **Logistic Regression** : Logistic Regression Used as a simple classifier due to computational effectiveness and ability to work well in text classification tasks [10]. This model, combined with TF-IDF features, provides high performance while remaining interpretable
- **Support Vector Machine (SVM)** :Used for robust classification of high-dimensional sparse feature space generated by TF-IDF vectorization. Thanks to the maximization of the margin between classes, SVM provides robust classification in text mining tasks [11].

2) Deep Learning model:

Bidirectional Long Short-Term Memory (BiLSTM): Used to extract sequential and contextual information from text. In contrast to ML models, BiLSTM processes text sequences in two directions - from left to right and vice versa - thus enabling the model to capture contextual dependencies based on surrounding words and phrases [12].

3) Transformer-Based Model:

DistilBERT(fine-tuned)

Used as a compact transformer-based architecture that preserves a significant amount of BERT-like contextual comprehension while being computationally less expensive. Due to the self-attention mechanism, DistilBERT is able to extract long-distance semantic dependencies from text. Fine-tuning was done for the multi-label toxic comment classification task[13].

D. Training Procedure and Hyperparameters

To conduct a fair comparison, all models were trained with the same train-test split and performance evaluation metrics were the same. Experiments were conducted with raw and preprocessed text datasets.

TABLE I
MODEL CONFIGURATIONS AND KEY HYPERPARAMETERS

Model	Key Hyperparameters
Logistic Regression	max_iter=1000
SVM	kernel=linear, C=1.0
BiLSTM	embedding_dim=128, hidden_units=64, batch_size=64, epochs=5
DistilBERT	learning_rate=2e-5, batch_size=16, epochs=1

E. Tools and Frameworks

The proposed system was implemented in Python using Google Colab as the development environment. Pandas and NumPy were used for data processing, while NLTK and Regular Expressions (Regex) were employed for text pre-processing. Matplotlib, Seaborn, and WordCloud were used for data visualization and exploratory analysis. Traditional machine learning models, including Logistic Regression and SVM, were implemented using Scikit-learn with TF-IDF feature extraction. The BiLSTM model was developed using TensorFlow/Keras, while the DistilBERT model was implemented using PyTorch and the Hugging Face Transformers library.

TABLE II
SOFTWARE ECOSYSTEM, LIBRARIES, AND FRAMEWORKS

Tool / Library	Primary Purpose in Framework
Python	Core Programming Language
Google Colab	(GPU Runtime)
Pandas	Structured Data Processing and Manipulation
NLTK	Text Preprocessing and Tokenization
Scikit-learn	Baseline ML Models and TF-IDF Vectorization
TensorFlow / Keras	BiLSTM Implementation and Training
PyTorch	Underlying Transformer Backend Environment
Transformers (HF)	DistilBERT Architecture and Fine-Tuning
Matplotlib / Seaborn	Evaluation Metrics Visualization

V. IMPLEMENTATION AND RESULTS

A. Experimental Setup

All experiments were conducted using Python in a Google Colab environment. The dataset was divided into training and testing subsets to ensure a fair evaluation of all models. To investigate the impact of preprocessing, each model was trained and evaluated using both raw text and preprocessed text versions of the dataset. Traditional machine learning models utilized TF-IDF feature representations, while deep learning and transformer-based models operated on tokenized text sequences. Identical dataset splits and evaluation criteria

were maintained across all experiments to ensure a consistent comparison.

B. Performance Metrics

In order to analyze model performance, multiple classification metrics were used.

a) *Accuracy*: Accuracy measures the proportion of correctly classified instances:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (10)$$

b) *Precision*: Precision measures the proportion of predicted positive samples that are actually positive:

$$Precision = \frac{TP}{TP + FP} \quad (11)$$

c) *Recall*: Recall measures the ability of the model to identify positive samples:

$$Recall = \frac{TP}{TP + FN} \quad (12)$$

d) *F1-Score*: The F1-score balances Precision and Recall:

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall} \quad (13)$$

e) *Macro F1-Score*: Since the dataset contains imbalanced toxicity labels, Macro F1-score was used as an important evaluation metric:

$$MacroF1 = \frac{1}{N} \sum_{i=1}^N F1_i \quad (14)$$

where N represents the number of toxicity classes.

Additionally, training time and inference time were recorded to compare computational efficiency among different model architectures.

C. Implementation Details

The pipeline consisted of data preprocessing, feature extraction, model training, and performance evaluation.

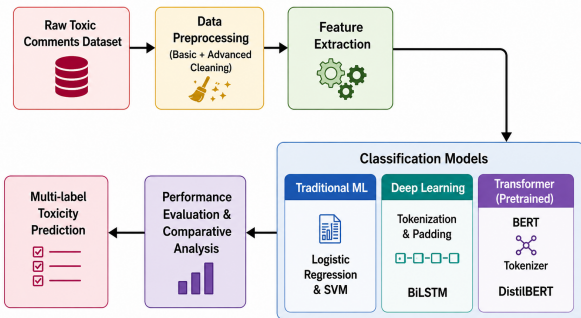


Fig. 7. End-to-end multi-label toxicity classification pipeline architecture

1) *Logistic Regression*: Logistic regression model was built using the Scikit-learn library with features extracted from TF-IDF representation of the toxic comment dataset. The Logistic Regression is an easy-to-compute baseline classifier and predict the likelihood of each toxicity category by applying the sigmoid activation function.

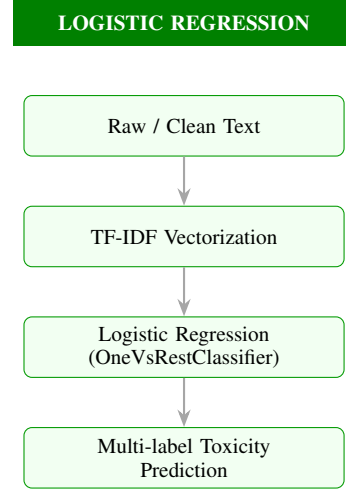


Fig. 8. System framework for the baseline Logistic Regression classification pipeline.

2) *Support Vector Machine (SVM)*: SVM was implemented in Scikit-learn using TF-IDF vector representation of text comments. SVM classifier was chosen because of its efficiency in working with high dimensional and sparse data and performing well in text classification.

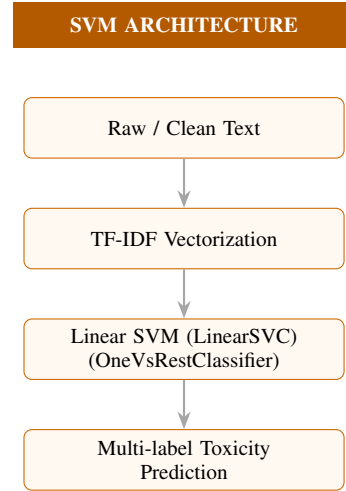


Fig. 9. Structural pipeline optimization utilizing Support Vector Machines.

3) *Bidirectional Long Short-Term Memory (BiLSTM)*: The BiLSTM model was implemented using TensorFlow/Keras. Text comments were preprocessed and encoded in numbers and padded to equal length prior to training. The model had an Embedding Layer, Bidirectional LSTM Layer, Global Max Pooling Layer, and Dense Output Layer with sigmoid activation for multi-class classification.

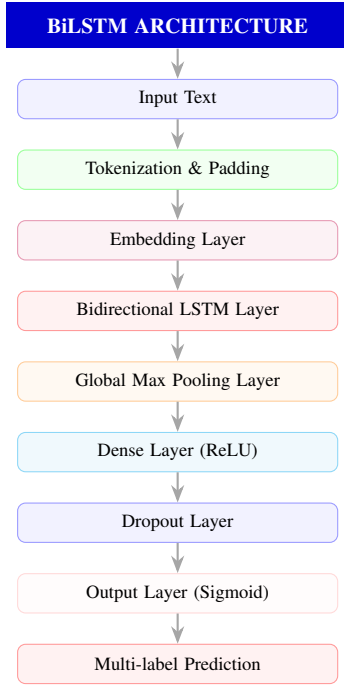


Fig. 10. Deep learning layer hierarchy for the BiLSTM neural pipeline.

4) *DistilBERT*: The DistilBERT model was implemented using the Hugging Face Transformers library with PyTorch framework. Pre-trained DistilBERT tokenizer encoded input comments to contextual tokens, which were used for fine-tuning.

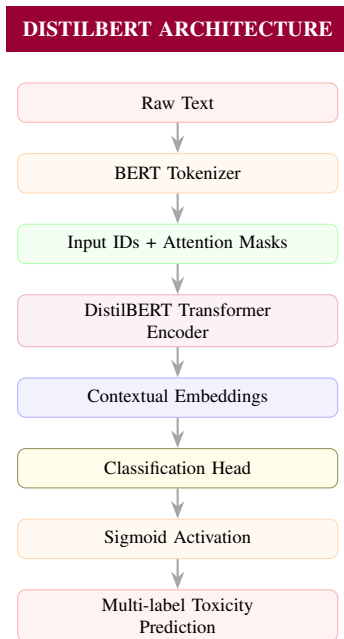


Fig. 11. Transformer-based model framework utilizing DistilBERT for sequence encoding.

D. Model Performance Comparison

The chart presents the comparative performance of all evaluated models. The results demonstrate the effectiveness of traditional machine learning, deep learning, and transformer-based approaches for toxic comment classification. Among the evaluated methods, the TF-IDF + SVM model achieved the highest overall performance, obtaining an accuracy of 92.09% and a Macro F1-score of 0.5944. The BiLSTM and DistilBERT models were capable of capturing contextual information but required significantly greater computational resources.

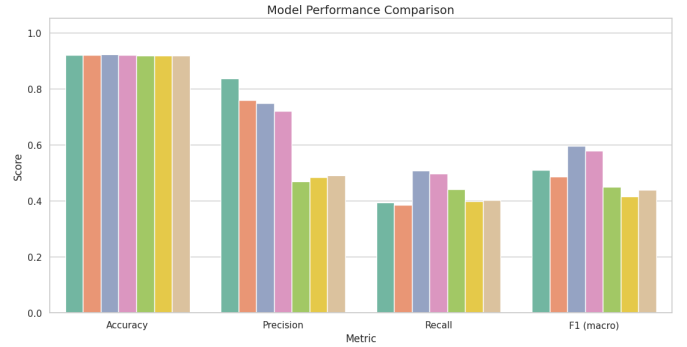


Fig. 12. Model Comparison

TABLE III
FINAL RESULTS SUMMARY

Model	Acc.	Prec.	Rec.	F1	Time (s)
TF-IDF + LR (raw)	0.9204	0.8366	0.3936	0.5099	17.9
TF-IDF + LR (adv.)	0.9189	0.7585	0.3845	0.4852	11.3
TF-IDF + SVM (raw)	0.9209	0.7476	0.5065	0.5944	12.7
TF-IDF + SVM (adv.)	0.9187	0.7193	0.4957	0.5780	6.6
LSTM – adv._clean	0.9175	0.4687	0.4416	0.4499	93.9
LSTM – Raw	0.9176	0.4846	0.3973	0.4150	47.0
BERT – Raw	0.9172	0.4849	0.4029	0.4377	1271.3

E. Confusion Matrix Analysis

The confusion matrices were created for analysis of the classification process of each model. It has been discovered that the models work fine when identifying the majority classes; meanwhile, classification errors are more likely for minority classes of toxicity since of class imbalance problem. The vast majority of misclassification cases were between toxic and non-toxic samples with ambiguous language and limited context. The confusion matrix has also shown that it is hard to detect rare labels like threat and identity-hate.

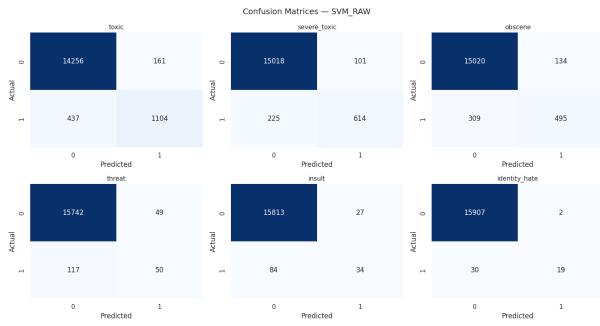


Fig. 13. Confusion Matrix for SVM (Raw)



Fig. 14. Confusion Matrix for SVM (Clean)

Figure 13 demonstrates the confusion matrix of the best performing raw-text model while Figure 14 presents the same but preprocessed one. The figures illustrate the difference that was made by preprocessing for classification performance.

F. Training and Inference Time Analysis

Training and inference times were measured in order to examine the computational efficiency. Classical machine learning models – Logistic Regression and SVM – needed much less time for training compared to BiLSTM and DistilBERT. Despite the fact that transformer-based models offered more complex contextual representation of the data, they were computationally costly both for training and inference processes. The results show that classical machine learning models can be considered a good choice because of their efficiency when dealing with toxic comments classification task.

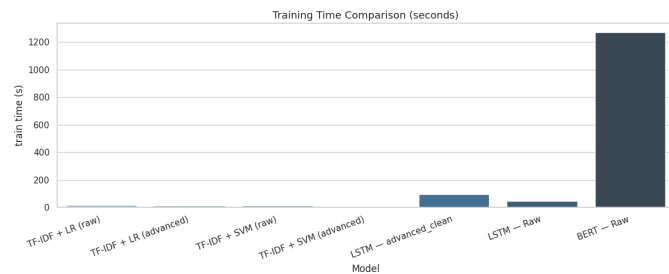


Fig. 15. Training time comparison figure

G. Effect of Preprocessing

In order to examine the impact of the text preprocessing, each model was tested on raw and cleaned datasets. The figures below demonstrate the overall comparison of Macro F1-scores while the following figures present model-specific results.

• Impact on Logistic Regression

As can be seen from the results, the model trained on raw text has a higher Macro F1-score compared to the model trained on cleaned text. The reason for the decreased performance after preprocessing is the fact that some patterns in texts, punctuation, and informal expressions provided useful discriminating features. Since TF-IDF algorithm heavily depends on word statistics, too much preprocessing leads to a loss of discriminating features.

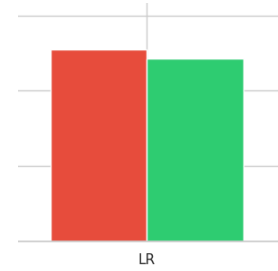


Fig. 16. Impact on Logistic Regression

• Impact on Support Vector Machine (SVM)

Similar situation happened with SVM classifier. The version of the model trained on raw texts demonstrated the highest overall performance among all other models with Macro F1-score equal to 0.59. While preprocessing eliminated the noise from the texts, it also made the model lose the information used for separating classes in high-dimensional TF-IDF space.



Fig. 17. Impact on SVM

• Impact on BiLSTM

Unlike the classical machine learning models, the BiLSTM model was positively impacted by preprocessing. The cleanedtext variant scored higher Macro F1-score compared to the raw-text one. Preprocessing allowed to get rid of noise elements like extra characters, repeated symbols and other irrelevant tokens, which made possible for the model to identify relevant sequence patterns and relationships in the comments.

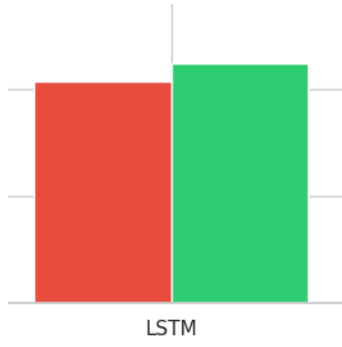


Fig. 18. Impact on BiLSTM

• Transformer-Based Model (DistilBERT)

Further preprocessing was not applied to DistilBERT. Transformer-based models are trained on extensive datasets and use subword tokenization methods, which means they are meant to work with natural language data in its original state. Too much preprocessing can delete contextual hints, punctuation and other linguistic information needed by the model to understand text. Thus, DistilBERT was trained on the raw text dataset in order to preserve the knowledge gained during the pre-training stage.

In summary, the obtained results demonstrate that the impact of preprocessing greatly depends on the model used. Machine learning models worked slightly better on the raw text while the BiLSTM model was positively affected by text cleaning. This proves that the preprocessing method should be chosen according to the specifics of the model rather than applied universally.

H. Error Analysis

The error analysis was carried out via confusion matrices and examples of incorrectly classified comments in the six labels classification scenario. It appears that the vast majority of errors are caused by ambiguous and context-dependent comments, sarcastic comments and implicit toxicity.

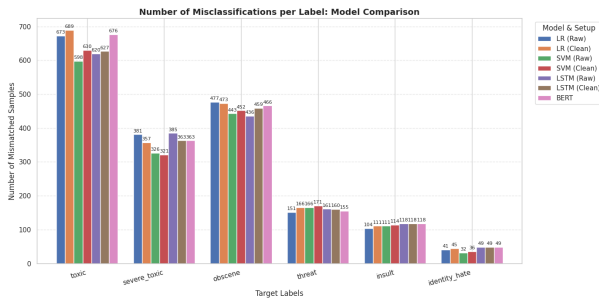


Fig. 19. Error Analysis

VI. DISCUSSION

A. Limitations

Although the classification process has proven to be efficient enough in terms of competitive performance, there are several

limitations of this work:

- **Class Imbalance:** The majority of toxicity categories significantly exceed the number of minority classes, such as 'threat' and 'identity-hate'.
- **Limited Context:** Short comments do not provide sufficient information about the context of the discussion, which makes it difficult to interpret and classify the content.
- **Computational Constraints:** Due to resource limitations, DistilBERT could only be trained for 1 epoch.
- **Overfitting Risks:** Imbalanced data resulted in some signs of overfitting.

B. Challenges

The analysis of toxic content demonstrates several linguistic and computational bottlenecks:

- **Implicit Toxicity Detection:** The absence of explicitly offensive language poses a challenge to the models that should detect the negative intent behind the messages.
- **Sarcasm and Ambiguity:** Sarcastic messages and ambiguity caused by the context affect the real intent, thus resulting in misclassification.
- **Context Deprivation:** The context plays a significant role in message interpretation; however, it is not always available.
- **Limitations in feature representation:** Classic models are based on surface text features and fail to detect deeper semantic patterns.

C. Possible Improvements

To solve the problems mentioned above, the following suggestions are made:

- **Class Balancing:** Use SMOTE, undersampling, or class weights in order to increase the performance of minority classes.
- **Data Augmentation:** Use methods like back translation, synonym substitution, or paraphrasing in order to enrich the dataset.
- **Extended Fine-Tuning:** Train transformers for multiple epochs using regularization and early stopping.
- **Hyperparameter Optimization:** Tune parameters such as learning rate, batch size, sequence length, and drop rate.
- **Bigger datasets:** Integrate multiplatform datasets of toxic content in order to increase generalization capabilities.
- **Advanced Architectures:** Evaluate larger models like RoBERTa, DeBERTa, or ELECTRA for superior contextual understanding.
- **Ensemble Learning:** Mix classic machine learning algorithms with transformer-based ones to utilize features and contextual data.
- **Context Enrichment:** Incorporate conversational thread history to better detect implicit toxicity and sarcasm.
- **Robust Evaluation:** Perform k-fold cross-validation.
- **Explainable AI:** Use SHAP or visualization of attention mechanism in order to make the model more transparent.

- **Deployment Optimization:** Utilize quantization, compression, or knowledge distillation to accelerate real-time inference.

VII. CONCLUSION

This paper conducted a comparative evaluation of classical machine learning, deep learning and transformer-based approaches for toxic comments classification based on the Jigsaw Toxic Comment Dataset. The effect of text preprocessing was studied by evaluating the performance of the models on both raw and preprocessed text.

It was shown that Support Vector Machine (SVM) outperformed other models on average. The combination of TF-IDF feature extraction with SVM classification was able to capture lexical patterns and word distributions typical for toxic language. Logistic Regression produced acceptable results, while BiLSTM showed high potential in learning of sequential information from data, yet it needs large amounts of data to reach its full capacity. Despite good contextual understanding of DistilBERT, its performance turned out to be worse than expected due to insufficient fine-tuning, as the model was trained for only one epoch.

Overall, the obtained results show that the performance of the models depends not only on the architecture complexity but also on the type of features used, preprocessing approach, dataset characteristics and training procedure. In the current experimental setup SVM proved to be the most efficient method.

Future Work

Possible future directions for improvement and development of this research include:

- Fine-tuning transformer-based models for extra epochs in order to improve adaptation of pretrained embeddings to the problem of toxic comments classification.
- Hyperparameters tuning for all models in order to further enhance their performance.
- Evaluation of advanced architectures of transformers such as BERT, RoBERTa and DeBERTa.
- Extending the system to support multilingual toxic comment detection across different languages and online communities.
- Implementing explainable AI methods to improve the interpretability of models and the understanding of the prediction process.
- Creating a real-time content moderation system which could be implemented in social media platforms, discussion forums and other online communication systems.

REFERENCES

- [1] S. Zaheri, J. Leath, and D. Stroud, "Toxic comment classification," *SMU Data Science Review*, vol. 3, no. 1, p. 13, 2020.
- [2] D. Androcec, "Machine learning methods for toxic comment classification: a systematic review," *Acta Universitatis Sapientiae, Informatica*, vol. 12, no. 2, pp. 205–216, 2020.
- [3] Google, "Jigsaw toxicity prediction dataset." https://huggingface.co/datasets/google/jigsaw_toxicity_pred, 2024. Hugging Face Dataset Repository, Accessed: 2026-06-22.
- [4] V. Maslej-Krešňáková, M. Sarnovský, P. Butka, and K. Machová, "Comparison of deep learning models and various text pre-processing techniques for the toxic comments classification," *Applied Sciences*, vol. 10, no. 23, p. 8631, 2020.
- [5] J. W. Kim, A. Guess, B. Nyhan, and J. Reifler, "The distorting prism of social media: How self-selection and exposure to incivility fuel online comment toxicity," *Journal of Communication*, vol. 71, no. 6, pp. 922–946, 2021.
- [6] J. Sarker, A. K. Turzo, M. Dong, and A. Bosu, "Automated identification of toxic code reviews using toxicr," *ACM Transactions on Software Engineering and Methodology*, vol. 32, no. 5, pp. 1–32, 2023.
- [7] Q. Liu, J. Wang, D. Zhang, Y. Yang, and N. Wang, "Text features extraction based on tf-idf associating semantic," in *2018 IEEE 4th international conference on computer and communications (ICCC)*, pp. 2338–2343, IEEE, 2018.
- [8] Z. He, Z. Wang, W. Wei, S. Feng, X. Mao, and S. Jiang, "A survey on recent advances in sequence labeling from deep learning models," *arXiv preprint arXiv:2011.06727*, 2020.
- [9] A. Rahali and M. A. Akhloufi, "End-to-end transformer-based models in textual-based nlp," *Ai*, vol. 4, no. 1, pp. 54–110, 2023.
- [10] M. P. LaValley, "Logistic regression," *Circulation*, vol. 117, no. 18, pp. 2395–2399, 2008.
- [11] D. A. Pisner and D. M. Schnyer, "Support vector machine," in *Machine learning*, pp. 101–121, Elsevier, 2020.
- [12] L. Enamoto, A. R. Santos, R. Maia, L. Weigang, and G. P. R. Filho, "Multi-label legal text classification with bilstm and attention," *International Journal of Computer Applications in Technology*, vol. 68, no. 4, pp. 369–378, 2022.
- [13] R. K. Piu, K. R. Hossain, N. H. Sabab, and R. B. M. A. Rafi, *Hatred Message Identification using DistilBERT*. PhD thesis, Ph. D. thesis, United International University, 2020.
- [14] A. Bonetti, M. Martínez-Sober, J. C. Torres, J. M. Vega, S. Pellerin, and J. Vila-Francés, "Comparison between machine learning and deep learning approaches for the detection of toxic comments on social networks," *Applied Sciences*, vol. 13, no. 10, p. 6038, 2023.